

Lab 4: VibeCoding - Chat History

Q*bert Repair Lab (SETAPESU26/35_qbert) - conversation with Claude

Exported up to and including the request for this PDF

User - Turn 1

[Attached: Lab_4_VibeCoding_Student_handout.pdf - Lab 4: VibeCoding. Objective: use Vibe Coding tools to fix the broken code and add features, prompting to get the job done in three to four attempts. Clone the assigned repo, make changes, push to your own repo. Deliverables: before/after videos, updated code, chat history exported as doc/pdf. Do not raise a PR to SETAPESU26.]

https://github.com/SETAPESU26/35_qbert

Claude - Turn 1

(Claude fetched the repo README and game.py before answering.)

I read both the handout and the repo. It's a Pygame Q*bert clone with one bug (Task 1) and three empty hooks (Tasks 2-4). You can do the whole thing in a few prompts. Here's what I found and what I'd change.

Before you start

Clone or fork the repo, run `python game.py`, and record your 10-second "before" video. The skewed pyramid is visible from the first frame.

Task 1: the bug

The problem is in `cube_center`:

```
# broken
WIDTH / 2 + (col - row // 2) * CUBE_W
# fixed
WIDTH / 2 + (col - row / 2) * CUBE_W
```

`row // 2` is integer division, so odd rows get rounded and shift sideways instead of being centred by half a cube. True division (`/`) fixes it.

Tasks 2-4: implementations

Task 2, `cube_palette(level)`: one palette per level, cycling with modulo so it never runs out.

```
LEVEL_PALETTES = [
    [(90, 160, 220), (190, 120, 70), (100, 210, 140)],
    [(200, 80, 120), (240, 200, 60), (80, 200, 200)],
    [(120, 90, 200), (230, 130, 50), (170, 230, 90)],
]

def cube_palette(level):
    return LEVEL_PALETTES[(level - 1) % len(LEVEL_PALETTES)]
```

Task 3, `on_cube_completed(cell)`: a short flash on the finished cube. The function only receives `cell`, so it records an expiry time and `draw_cube` reads it.

```
FLASHES = {}

def on_cube_completed(cell):
    FLASHES[cell] = pygame.time.get_ticks() + 250
```

In `draw_cube`, right after `color = colors[self.stages[cell]]`, add:

```
if FLASHES.get(cell, 0) > pygame.time.get_ticks():
    color = (255, 255, 255)
```

Task 4, bonus_life_threshold():

```
def bonus_life_threshold():  
    return 1000
```

A bug you'll hit after Task 4

`reset()` sets `self.bonus_awarded = 0` every time, including `reset(full=False)` between levels. Once your score is at 1000 or more, each new level will award a free life on the first frame. To fix it, move that line under the `if full:` block:

```
def reset(self, full=True):  
    if full:  
        self.level, self.score, self.lives = 1, 0, 3  
        self.bonus_awarded = 0
```

Add `self.bonus_awarded = 0` to `__init__` before `self.reset()` as well, or just keep the line in `reset` and guard it with `if full`. Either way the counter then persists across levels.

Prompting for the lab

The handout wants this done in 3-4 attempts, so paste the full `game.py` plus the relevant README task text each time:

- Fix the projection bug. Paste the Task 1 description and ask for the minimal change.
- Implement `cube_palette`, `on_cube_completed` and `bonus_life_threshold` together, with the README wording for each. Ask it not to change anything else.
- Ask it to review the bonus-life logic across level transitions. This should surface the `bonus_awarded` issue.
- Fix anything you found while playtesting.

For submission, take your "after" video after level 2 starts so it shows the new palette, a cube flash and the fixed pyramid. Push to your own repo under Lab-4/, with the videos, updated code and exported chat, and don't open a PR to SETAPESU26.

I can put the full updated `game.py` in one file if you'd like to diff it against your own version.

User - Turn 2

[Attached excerpt from previous Claude message: "Clone or fork the repo, run python game.py, and record your 10-second 'before' video. The skewed pyramid is visible from the first frame."]

describe the errors as they appear in the 'before' game so that i make to capture it in my before vedio

Claude - Turn 2

I worked this out by reading the code, not by running the game, so check it against your screen. Here's what the broken `cube_center` produces.

What you'll see on screen

1. The pyramid is lopsided. The `row // 2` rounding shifts every odd row (1, 3, 5) about half a cube (36 px) to the right. Rows 0, 2, 4 and 6 stay centred. So:

- The right edge zigzags outward and the left edge steps inward, and the shape leans right instead of forming a clean triangle.
- Row 1 is the clearest tell. Its two cubes sit at `x=360` and `x=432`, so the top cube is directly above the left cube instead of nestled between the two.

- Odd-row cubes stack tip-to-tip under the cube above, so the cubes look like a skewed grid instead of interlocking isometric blocks. You'll see small diamond-shaped gaps between them.

2. Hops go in inconsistent directions. Every hop should be a clean diagonal, but the skew makes some hops go straight down or sideways:

- From the top cube, DOWN hops straight down and RIGHT hops about 72 px right, where a correct pyramid gives a symmetric diagonal each way.
- From row 1, the pattern flips: DOWN hops 72 px left and RIGHT hops straight down.
- LEFT on the top cube (the "off the edge" hop) goes straight up instead of up-left, then Q*bert falls.

A 10-second script for the video

- **0-2 s:** Hold still on the starting frame so the leaning, zigzag pyramid is clear.
- **2-7 s:** Move right away, since a red ball spawns on the top cube around 2 s and costs a life if you're still standing there. Press DOWN, then RIGHT, then DOWN again so the straight-down and 72-px-sideways hops show up. Hop twice on one cube to show it going blue, then brown, then green with no flash.
- **7-10 s:** Press LEFT on the top cube to show the odd fall-off, or let Coily (the purple ball, spawns at ~3 s) appear and chase you.

Missing features to show

These are hard to demo in 10 seconds, so mention them in your chat or README notes instead:

- All cubes use the same blue/brown/green palette (cube_palette returns nothing), and it never changes on later levels.
- A completed cube turns green with no flash or bonus.
- Lives never go up, because bonus lives are disabled.

If you'd like, I can also write a short "before" description to paste into your Lab-4 notes.

User - Turn 3

[Attached excerpt from previous Claude message: the Task 2 block containing LEVEL_PALETTES and def cube_palette(level): ...]

which part of the code ie under which function do i insert this code block?

Claude - Turn 3

This goes at the top of game.py, in the section that holds the three empty stub functions. It isn't inside any class or other function.

Where each piece goes

1. LEVEL_PALETTES is a module-level constant. Put it right after DEFAULT_PALETTE (line 9), which holds the other constants.

2. cube_palette(level) replaces the existing empty stub. Delete the pass and the old body, then paste the new one in.

Before

```
DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]

KEY_HOPS = {...}

def cube_palette(level):
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
```

pass

After

```
DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]

LEVEL_PALETTES = [
    [(90, 160, 220), (190, 120, 70), (100, 210, 140)],
    [(200, 80, 120), (240, 200, 60), (80, 200, 200)],
    [(120, 90, 200), (230, 130, 50), (170, 230, 90)],
]

KEY_HOPS = {...}

def cube_palette(level):
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
    return LEVEL_PALETTES[(level - 1) % len(LEVEL_PALETTES)]
```

Keep the docstring and only swap pass for the return line.

How it connects

You don't call `cube_palette` yourself. `Game.draw()` already does it every frame with `colors = cube_palette(self.level)` or `DEFAULT_PALETTE`. Level 1 uses the first palette, level 2 the second, and so on, wrapping back to the first after level 3. To see the change in your "after" video, clear a level and press Space.

User - Turn 4

[Attached excerpt from previous Claude message: FLASHES = {} / def on_cube_completed(cell): FLASHES[cell] = pygame.time.get_ticks() + 250 / and the draw_cube flash check]

is flashes supposed to be empty

Claude - Turn 4

Yes, `FLASHES = {}` is meant to start empty. It's a dictionary that fills up while the game runs.

How it works

- At startup nothing is flashing, so the dict is empty.
- When you finish a cube, `paint()` calls `on_cube_completed(cell)`, which adds an entry. For example, `FLASHES[(2, 1)] = 5230` means cube (2, 1) should flash until the clock reads 5230 ms.
- Every frame, `draw_cube` checks whether the cube has an entry with an expiry time still in the future. If it does, the cube is drawn white. Once 250 ms pass, the check fails and the cube goes back to green.

Entries are never deleted, but that's harmless. Expired ones fail the time check and the dict is at most 28 cells.

Placement

- `FLASHES = {}` goes at module level, next to `DEFAULT_PALETTE` and `LEVEL_PALETTES`, above the functions.
- `on_cube_completed` replaces the existing stub. Swap pass for the one `FLASHES[cell] = ...` line.
- The `if FLASHES.get(...)` block goes inside `Game.draw_cube`, right after the `color = colors[self.stages[cell]]` line.

`FLASHES.get(cell, 0)` returns 0 for cubes with no entry, so cubes that never finished don't flash.

User - Turn 5

give me this chat history up till this prompt as a pdf

Claude - Turn 5

Generated this PDF of the conversation (Turns 1-5).